Week-3

Structure of a Python Program, Elements of Python, Python Interpreter, Input-Output: Printing on screen, Reading data from keyboard, Python shell, Indentation. Atoms, Identifiers and keywords, Literals, Strings and Operators

---

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

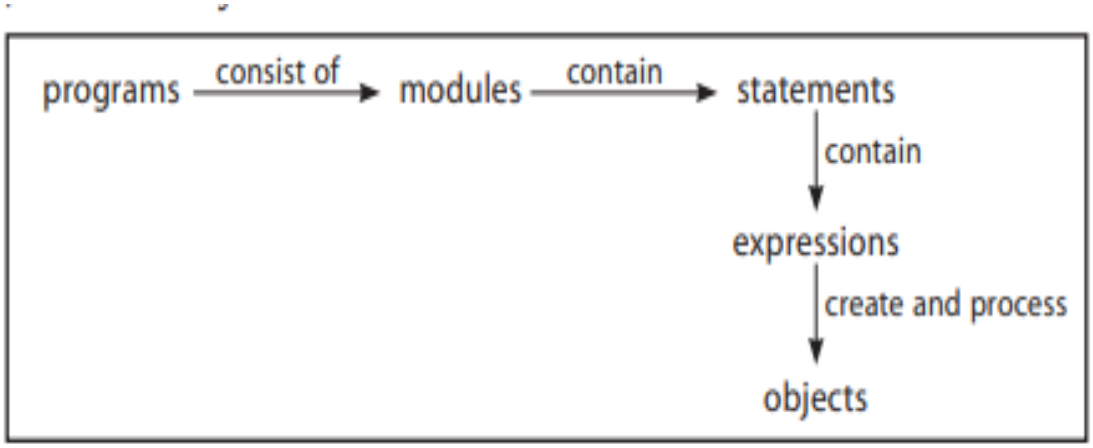
- web development (server-side),
- software development,
- mathematics,
- system scripting.

Structure of Python program

The typical structure of a python program include 3 parts

```
Import statements
// import statements are used to include library files to the python program
Function definitions
//This section includes the definitions of various functions written in a Python Program
Program statements
// This section includes the set of statements for solving the given problem.
```

A Python program constitutes several elements such as statements, expressions, functions, comments, etc., which are synchronized in the manner as shown below:



the several components that a Python program holds are:

- Expressions: An expression represents something, like a number, a string, or an element. Any value is an expression.
- Statements: Anything that does something is a statement. Any assignment to a variable or function call is a statement. Any value contained in that statement is an expression.
- Comments: Comments are the additional information provided against a statement or a chunk of code for the better clarity of the code. Interpreter ignores the comments and does not count them in commands. Symbols used for writing comments include Hash (#) or Triple Double Quotation marks (“””). Hash (#) is used in writing single-line comments that do not span multiple lines. Triple Quotation Marks (‘’’ or “””) are used to write multiple-line comments. Three triple quotation marks to start the comment and again three quotation marks to end the comment.
- Functions: Function is a set of instructions defined under a particular name, which once written can be called whenever and wherever required.
- Block(s): A block refers to a group of statements which are part of another statement or function. All statements inside a block are indented at the same level.

Elements of Python

Python has 4 components

1. IDLE (Python GUI): It is a cross platform GUI Integrated Development Environmental that is provided with Python for editing and running a Python programs. It is a bundled set of software’s tools such as Python Shell for Scripting programs and text editor for creating and modifying Python’s source code, an integrated interactive debugger for finding error, help system etc.

2. Module Docs: This component allows us to access the python documents such as build in modules, DLLs, libraries, packages etc.

3. Python (Command line): Python can also be access from the command line. Command line mode provide very less features in comparison to IDLE but it is fast.

4. Python Manual : This component include various documents related to Python such as : installation cum setup guide, tutorials, Python API , FAQs etc.

Python Interpreter

Python is a high level language. We know that computer understand machine language or binary language which is also known as low level language. The job to translate programming code written in high level language to low level language is done with the help of two type of softwares : compilers and interpreters.

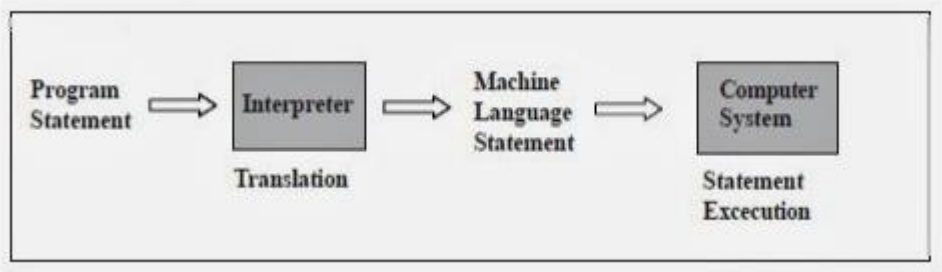
Working of Compiler

Compiler is a special type of software it first check the source code of the complete program for the syntax errors, if no error found in the program code then the source code of the program is converted to the machine code and finally the executive file of the program is created. The executive file can be executed by the CPU and can produce desired result as per the program requirements.



Working of Interpreter

Interpreter check the code of program line by line if no error is found it is converted into the machine code and simultaneously gets executed by the CPU.



An interpreter is a computer program, which coverts each high-level program statement into the machine code. This includes source code, pre-compiled code, and scripts. Both compiler and interpreters do the same job which is converting higher level programming language to machine code. However, a compiler will convert the code into machine code (create an exe) before program run. Interpreters convert code into machine code when the program is run.

Atoms

Atoms are the most basic elements of expressions. The simplest atoms are identifiers or literals. Forms enclosed in reverse quotes or in parentheses, brackets or braces are also categorized syntactically as atoms. The syntax for atoms is:

atom: identifier | literal | enclosure enclosure:
parenth_form|list_display|dict_display|string_conversion

Indentation

Indentation in Python refers to the (spaces and tabs) that are used at the beginning of a statement. The statements with the same indentation belong to the same group.

For Example :

```
a=2
 print(a)
if a==3 :
        print(“hello world”)
        print(“HELLO WORLD”)
else :
        print(“ bye world”) print(“ BYE WORLD”)
```

Identifier

Identifier is a name given to various programming elements such as a variable, function, class, module or any other object.

Following are the rules to create an identifier.

1. The allowed characters are a-z, A-Z, 0-9 and underscore (_)

2. It should begin with an alphabet or underscore

3. It should not be a keyword

4. It is case sensitive

5. No blank spaces are allowed.

6. It can be of any size

Valid identifiers examples : si, rate_of_interest, student1, ageStudent

Invalid identifier examples : rate of interest, 1student, @age

Keywords

Keywords are the identifiers which have a specific meaning in python, there are 33 keywords in python. These may vary from version to version.

False	None	True	and	as	assert	break	class
continue	def	del	elif	else	except	finally	for
from	global	if	import	in	is	lambda	nonlocal
not	or	pass	raise	return	try	while	with
yeild							

Variables and Types

In program to store values , we use variables to store data which can be manipulated by the computer program.

Every variable has:

A. Name and memory location where it is stored.
A variable name:
1. Can be of any size
2. Have allowed characters, which are a-z, A-Z, 0-9 and underscore (_)
3. should begin with an alphabet or underscore
4. should not be a keyword
It is a good practice to follow these identifier naming conventions:
1. Variable name should be meaningful and short
2. Generally, they are written in lower case letters

B. A type or datatype which specify the nature of variable (what type of values can be stored in that variable and what operations can be performed)

We can check the type of the variable by using type command

```
>>> type(variable_name)
```

C. A value

Various datatypes available in python are as follow :

1. Number Number data type stores Numerical Values.
2. These are of three different types:
   a) Integer & Long
   b) Float/floating point
   c) Complex
   1.1 Integers are the whole numbers consisting of + or – sign like 100000, -99, 0, 17.
   e.g. age=19 salary=20000
   1.2 Floating Point: Numbers with fractions or decimal point are called floating point numbers.
   A floating point number will consist of sign (+,-) and a decimal sign(.)
   e.g. temperature= -21.9, growth_rate= 0.98333328
   The floating-point numbers can be represented in scientific notation such as -2.0X 10^5 will be represented as -2.0e5
   2.0X10^{-5} will be 2.0E-5
   1.3 Complex: Complex number is made up of two floating point values, one each for real and imaginary part. For accessing different parts of a variable x we will

use x.real and x.imag. Imaginary part of the number is represented by j instead of i, so 1+0j denotes zero imaginary part.

Example

```
>>> x = 1+0j
>>> print x.real,x.imag

1.0 0.0
```

2. None :This is special data type with single value. It is used to signify the absence of value/false in a situation. It is represented by None.

3. Sequence :A sequence is an ordered collection of items, indexed by positive integers.

Three types of sequence data type available in Python are Strings, Lists & Tuples.

3.1 String: is an ordered sequence of letters/characters.

They are enclosed in single quotes (‘’) or double (“”).

Example

```
>>> a = 'Ram'
>>>a=”Ram”
```

3.2 Lists: List is also a sequence of values of any type.

Values in the list are called elements / items.

The items of list are accessed with the help of index (index start from 0).

List is enclosed in square brackets.

Example Student = [“Ajay”, 567, “CS”]

3.3 Tuples: Tuples are a sequence of values of any type, and are indexed by integers.

They are immutable i.e. we cannot change the value of items of tuples .

Tuples are enclosed in ().

Student = (“Ajay”, 567, “CS”)

4.Sets Set is an unordered collection of values, of any type, with no duplicate entry. Sets are immutable.

Example s = set ([1,2,3,4])

5. Dictionaries: Dictionaries store a key – value pairs, which are accessed using key. Dictionary is enclosed in curly brackets.

Example d = {1:'a', 2:'b', 3:'c'}

Literals

A literal is a constant that appear directly in a program and this value does not change during the program execution i.e. remain fixed.

Example:

Num1=5

Principle_amount= 5000.00

Here 5 and 5000.00 are literals.

Python support the following literals

- Numeric literals
- String Literals
- Special Literals
- Collection Literals

Operators and Operands

Operators are special symbols which represents specific operations. They are applied on operand(s), which can be values or variables.

Operators are classified into two types based on the working of operands.

- Unary operator: Unary operator is an operator which works on single operand.
- Binary operator: Binary operator is an operator which works on two operand.

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Arithmetic operators:

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Meaning	Example
+	Add two operands or unary plus	x + y+ 2
-	Subtract right operand from the left or unary minus	x - y- 2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y (x to the power y)

Comparison operators: Comparison operators are used to compare values. It returns either `True` or `False` according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

Logical Operators:

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
Or	Returns True if one of the statements is true	x < 5 or x < 4
Not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Assignment operators

Assignment operators are used in Python to assign values to variables. a=5 is a simple assignment operator that assigns the value 5 on the right to the variable a on the left.There are various compound operators in Python like a+=5 that adds to the variable and later assigns the same. It is equivalent to a=a+5

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5

&=	x &= 5	x = x & 5
|=	x |= 5	x = x | 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Bitwise Operators

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

X	y	x & y	x | y	x ^ y
0	0	0	0	0
0	1	0	1	1

1	0	0	1	1
1	1	1	1	0

Bitwise And

Let us consider that we have 2 variables op1 and op2 with values as follows:

```
Op1 = 0000 1101
Op2 = 0001 1001
```

The result of the AND operation on variables op1 and op2 will be

```
Result = 0000 1001
```

Bitwise Or

```
Op1 = 0000 1101
Op2 = 0001 1001
```

The result of the OR operation on variables op1 and op2 will be

```
Result = 0001 1101
```

Bitwise Xor

```
Op1 = 0000 1101
Op2 = 0001 1001
```

The result of the OR operation on variables op1 and op2 will be

```
Result = 0001 0100
```

Bitwise shift operators

The bitwise shift operators are used to move/shift the bit patterns either to the left or right side.

```
Operand << n (Left Shift)
Operand >> n (Right Shift)
```

Here,

- an operand is an integer expression on which we have to perform the shift operation.
- 'n' is the total number of bit positions that we have to shift in the integer expression.

The left shift operation will shift the 'n' number of bits to the left side. The leftmost bits in the expression will be popped out, and n bits with the value 0 will be filled on the right side.

The right shift operation will shift the 'n' number of bits to the right side. The rightmost 'n' bits in the expression will be popped out, and the value 0 will be filled on the left side.

Bitwise complement operator

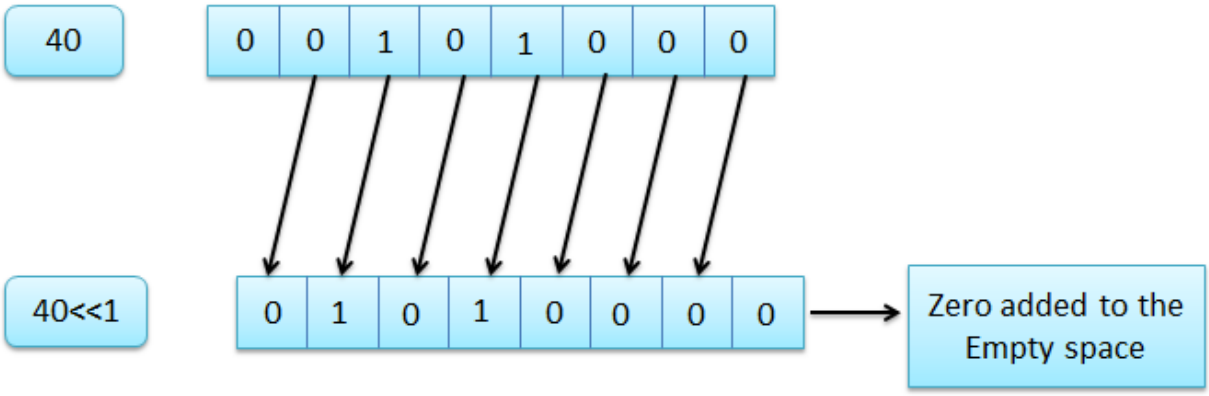
The bitwise complement is also called as one's complement operator since it always takes only one value or an operand. It is a unary operator.

When we perform complement on any bits, all the 1's become 0's and vice versa.

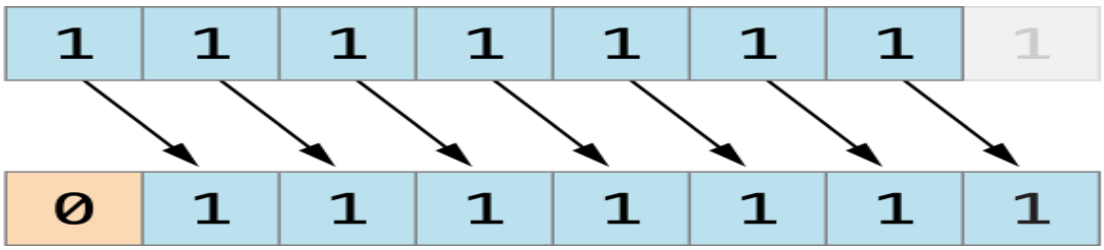
If we have an integer expression that contains 0000 1111 then after performing bitwise complement operation the value will become 1111 0000.

Bitwise complement operator is denoted by symbol tilde (~).

Left shift



Right Shift



Special operators

Python language offers some special types of operators like the identity operator or the membership operator. They are described below with examples.

Identity operators

`is` and `is not` are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
Is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Example :Identity operators in Python

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
# Output: False
print(x1 isnot y1)

# Output: True
print(x2 is y2)
```

Membership operators

`in` and `not in` are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
In	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Example : Membership operators in Python

```
x = 'Hello world'
# Output: True
print('H'in x)

# Output: True
print('hello'notin x)
```

Input/Output statements

Input Statement:

Python supports an input() function to accept input from user.

Syntax:

input('prompt')

where, prompt is a string that is displayed on the string at the time of taking input.

Example 1: Taking input from the user with a message.

```
# Taking input from the user
name = input("Enter your name: ")
```

Output:

Enter your name: GITAM

The value in name is GITAM.

Example 2: By default **input()** function takes the user's input in a string. So, to take the input in the form of int, we need to use **int()** along with input function.

```
# Taking input from the user as integer
num = int(input("Enter a number: "))
num1 = num + 1
```

Output:

Enter a number: 25

Value of

num=25

num1=26

Displaying Output

Python provides the print() function to display output to the console.

Syntax:

print(value(s), sep= ' ', end = '\n', file=file, flush=flush)

Parameters:
value(s) : *Any value, and Will be converted to string before printed.*
sep='separator' : *(Optional) Specify how to separate the objects, if there is more than one.Default :' '*
end='end': *(Optional) Specify what to print at the end.Default : '\n'*
file : *(Optional) An object with a write method. Default :sys.stdout*
flush : *(Optional) A Boolean, specifying if the output is flushed (True) or buffered (False). Default: False*

Returns: *It returns output to the screen.*

Example:

1.print("Hello")

Output:Hello

2.# code for disabling the softspace feature

print('A', 'B', 'C', sep ='')

Output:ABC

3.# using end argument

print("Python", end = '@')

print("Programming Language ")

output:

Python@ Programming Language

Strings

Strings are sequences of characters. Strings can contain alphabets, numbers and some special characters.

Strings can be created by enclosing characters inside a single quote or double-quotes.

Example:

S= ‘hello’

a= “python programming”

Multiple lines can be done using python’s triple quotes. It is also used for long comments in code. Special characters like TABs, NEWLINEs can also be used within the triple quotes. We can use either triple single or double quotes.

Example:

str='''python is an interpreted language,

python is dynamic typed language,

python is user friendly language'''

print(str)

output:

python is an interpreted language,

python is dynamic typed language,

python is user friendly language

Examples:

my_string = 'Hello'

```
print(my_string)

my_string = "Hello"
print(my_string)

my_string = '''Hello'''
print(my_string)

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
        the world of Python"""
print(my_string)
```

When you run the program, the output will be:

```
Hello
Hello
Hello
Hello, welcome to
        the world of Python
```

Accessing characters in Python:

- We can access the individual characters in a string through *indexing*.
- The positions in a string are numbered from the left, starting with 0.



>>>txt= ‘Hello World’

>>>txt[6]

‘W’

- In a string of *n* characters, the last character is at position *n-1* since we start counting with 0.

String Slicing:

We can also access a contiguous sequence of characters, called a *substring*, through a process called *slicing*

- Slicing:
  <string>[<start>:<end>]
- start and end should both be integers
- The slice contains the substring beginning at position start and runs up to but doesn’t include the position end.



>>>s[2:7] # starting at 2, up to but not including 7

llo W

>>>s[6:10]

Worl

Slice from start: By leaving out the start index the range will start at the first character

>>>s[:5]

Hello

>>>s[:7]

Hello W

Slice to end: By leaving out the *end* index, the range will go to the end

>>>s[6:]

World

>>>s[2:]

llo World

Negative Indexing:

- As an alternative, Python supports using negative numbers to index into a string: -1 means the last char, -2 is the next to last, and so on. **We can index from the right side using negative indexes**

H	e	l	l	o		W	o	r	l	d
-11	**-10**	**-9**	**-8**	**-7**	**-6**	**-5**	**-4**	**-3**	**-2**	**-1**

>>>txt= 'Hello World'

>>>txt[-4]

'o'

>>>txt[-5]

'W'

>>>txt[-5:-1]

Worl

>>>txt[-5:]

World

>>>txt[:-1]

Hello Worl

>>>txt[-5:-2]

Wor

>>>txt[:-4] #slice from start to -3(-4 not included)

Hello W

>>>txt[-5:] #begin slice with -5 to the end

World

#Accessing string characters in Python

str = 'GITAM University'

print('str = ', str)

#first character

print('str[0] = ', str[0])

#last character

print('str[-1] = ', str[-1])

#slicing 2nd to 5th character

print('str[1:5] = ', str[1:5])

#slicing 6th to 2nd last character

print('str[5:-2] = ', str[5:-2])

When we run the above program, we get the following output:

str = GITAM University

str[0] = G

str[-1] = y

str[1:5] = ITAM

str[5:-2] = Universi

Modify/Delete a String in Python

Strings are immutable. It suggests that once a String binds to a variable, it can't be modified. In other words, This means that elements of a string cannot be changed once they have been assigned. We can simply reassign different strings to the same name.If you want to update the String, then re-assign a new String value to the same variable.

```
>>> my_string = 'GITAM'
>>> my_string[3] = 'a'
...
TypeError: 'str' object does not support item assignment
>>> my_string = 'Python'
>>> my_string
'Python'
```

Similarly, we cannot modify the Strings by deleting some characters from it. Instead, we can remove the Strings altogether by using the 'del' command.

```
str1 = "Python"
del str1[1]
# TypeError: 'str' object doesn't support item deletion
del str1
print(str1)
# NameError: name 'str1' is not defined
```

Operations on strings

Assume string variable **a** holds 'Gitam' and variable **b** holds 'University', then −

Operator	Description	Example
+	Concatenation - Adds values on either side of the operator	a + b will give GitamUniversity
*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give -GitamGitam
[]	Slice - Gives the character from the given index	a[1] will give i
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ita
in	Membership - Returns true if a character exists in the given string	‘t’ in a will give True
not in	Membership - Returns true if a character does not exist in the given string	‘B’ not in a will give True

Examples:

Concatenation of Two or More Strings: Joining of two or more strings into a single one is called concatenation. The + operator does this in Python. Simply writing two string literals together also concatenates them.

```
str1 = 'Hello'

str2 ='World!'

# using +

print('str1 + str2 = ', str1 + str2)
```

When we run the above program, we get the following output:

```
str1 + str2 =  HelloWorld!
```

Writing two string literals together also concatenates them like + operator.

If we want to concatenate strings in different lines, we can use parentheses.

```
>>> # two string literals together
>>> 'Hello "World!'
'Hello World!'


>>> # using parentheses
>>> s = ('Hello '
...      'World')
>>> s='Hello World'
```

2.Repetition : The * operator can be used to repeat the string for a given number of times.

```
str1 = 'Hello'
# using *
print('str1 * 3 =', str1 * 3)
```

When we run the above program, we get the following output:

```
str1 * 3 = HelloHelloHello
```